

Security Audit

AI Protocol (Artificial Intelligence)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	11
Audit Resources	11
Dependencies	11
Severity Definitions	12
Status Definitions	13
Audit Findings	14
Centralisation	23
Conclusion	24
Our Methodology	25
Disclaimers	27
About Hashlock	28

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The AI Protocol team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

AI Protocol is a decentralized ecosystem for launching and scaling persistent onchain AI economies. It provides infrastructure, programmable logic, and governance standards so autonomous agents, applications, and models can operate continuously, coordinate economically, and persist independently. The protocol supports creating, owning, monetizing, and exchanging tokenized AI components with verifiable utility, provenance, and value, enabling transparent ownership and auditable lifecycle history. By combining onchain tokenization with multimodal AI capabilities, the system supports expressive, responsive interactions while remaining composable across a broader decentralized AI economy. This framework allows developers, creators, and users to participate through open standards, transparent incentives, and protocol-native mechanisms that support sustainable, long-term value creation.

Project Name: AI Protocol

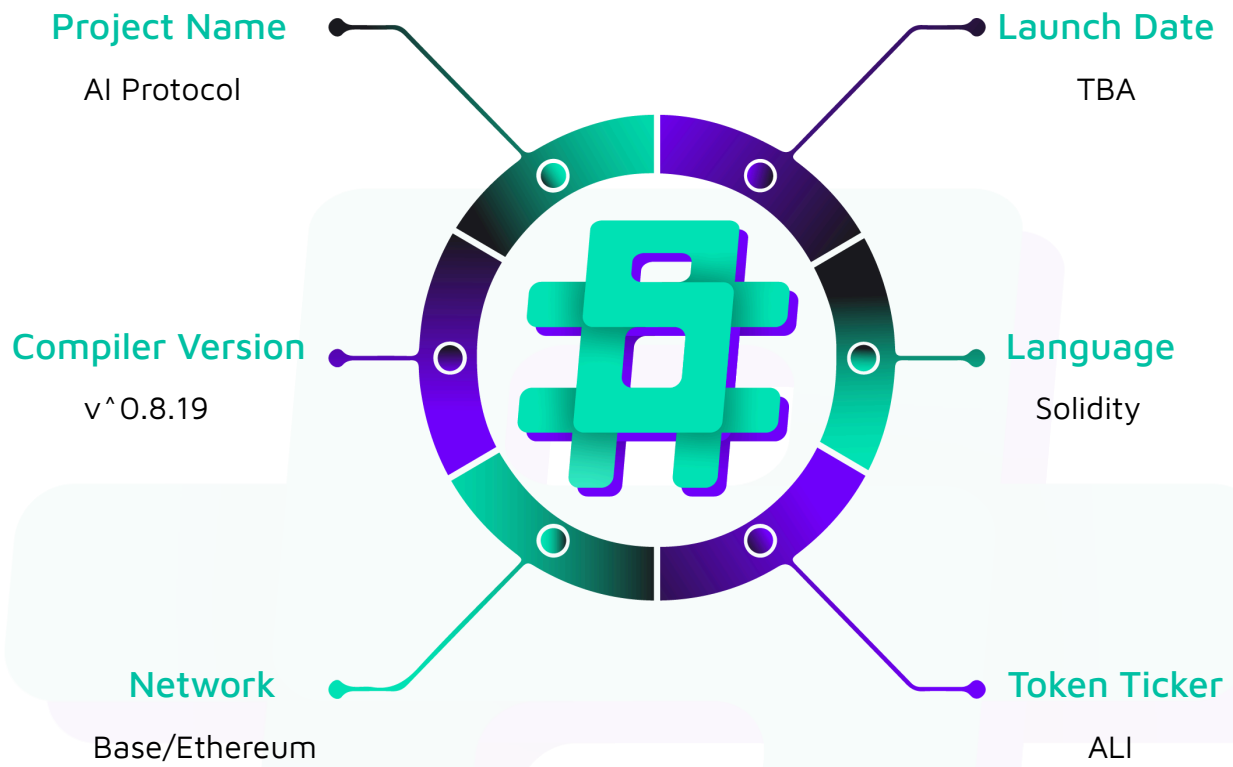
Project Type: AI

Compiler Version: ^0.8.19

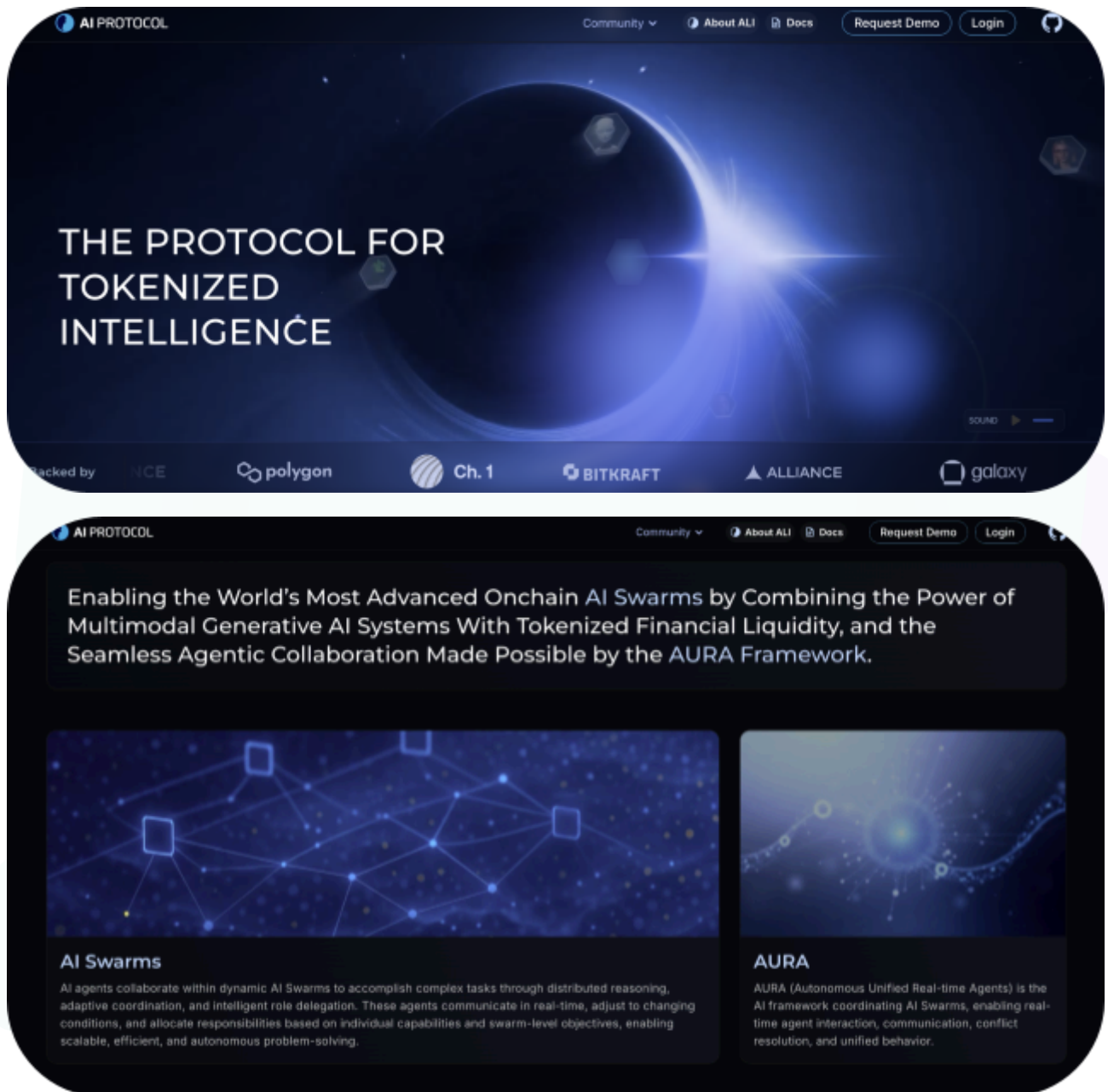
Website: <https://www.aiprotocol.ai/>

Logo:



Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the Solidity code within the AI Protocol project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	AI Protocol Smart Contracts
Network	Base/Ethereum
Audit Date	January, 2026
Contract 1	GenesisTokenFactoryV1
Contract 2	UniswapV4SwapFeeHookV1
Contract 3	V4HookFeeDistributorV2
Contract 4	UniswapV4PoolInitializer
Audited GitHub Commit Hash	e42b763933d3aca8d1e52dc92ef5a942e33ef0b3
Fix Review GitHub Commit Hash	59ca75e9fa4ff60a97fb9ed1f2740f76915ca411

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

2 High severity vulnerabilities

2 Medium severity vulnerabilities

3 Low severity vulnerabilities

4 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
GenesisTokenFactoryV1.sol - Allows the admin to: - Update core config - Update allocation shares - Update LP allocations - Grant roles, update metadata on launched tokens - Launch an economy on behalf of a pod owner - Allows users to: - Launch their economy via 'eip712LaunchEconomy' only if they are the podOwner, the feature is enabled, and a valid signature from an authorized signer is provided - Do read-only queries	Contract achieves this functionality.
UniswapV4SwapFeeHookV1.sol - Allows the admin to: - Configure fee token recipient + buy/sell bps - Remove config - Allows users to: - Read-only getFeeConfiguration	Contract achieves this functionality.
V4HookFeeDistributorV2.sol - Allows the admin to: - Set per-POD fee recipients	Contract achieves this functionality.

- Set fee split - Set distribution period - Set factory address - Distribute accrued fees to recipients - Record fees - Allows users to: - Claim their accrued fees - Do read-only queries	
UniswapV4PoolInitializer.sol - Allows the admin to: - Wire Uniswap v4/Permit2 addresses once via internal initializer	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the AI Protocol project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the AI Protocol project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] UniswapV4SwapFeeHookV1#updateFeeConfiguration, deleteFeeConfiguration - Stale infinite approvals to fee recipients enable unauthorized fee-token withdrawals

Description

The hook grants unlimited ERC20 allowance to the fee recipient and does not revoke the old recipient when the recipient is updated.

Vulnerability Details

This hook uses infinite approve as a standing capability for the fee recipient to pull fee tokens from the hook contract. Whoever controls the fee recipient controls a persistent transfer right over hook-held balances. Moreover, when you update the recipient, the old recipient's allowance is not revoked. As a result, any previously configured fee recipient can retain indefinite pull-rights even after being replaced.

```
// in __updateFeeConfiguration()

feeTokens[feeToken] = FeeConfig(feeRecipient, buyFeeBps, sellFeeBps);

ERC20(Currency.unwrap(feeToken)).approve(address(feeRecipient), type(uint256).max);

...

// in deleteFeeConfiguration()

ERC20(Currency.unwrap(feeToken)).approve(address(feeTokens[feeToken].feeRecipient), 0);

delete feeTokens[feeToken];
```

Impact

If a fee recipient is compromised, it can drain fee tokens from the hook contract without needing any swap to occur at that moment. Separately, even if the current recipient is

safe, a previously configured recipient may still be able to drain funds due to the lack of revocation on update.

Recommendation

Revoke the old recipient before approving the new one, and avoid infinite approvals where possible (approve exactly what you need per operation, or use time constrained approvals).

Status

Resolved

[H-02] UniswapV4PoolInitializer#_createPoolAndAddLiquidity - Lack of safe-approve pattern can brick launches

Description

Token approvals are performed without SafeERC20 patterns and without checking the `bool` return value.

Vulnerability Details

```
ERC20(Currency.unwrap(key.currency0)).approve(address(permit2), amount0);  
ERC20(Currency.unwrap(key.currency1)).approve(address(permit2), amount1);  
  
permit2.approve(Currency.unwrap(key.currency0), address(positionManager), amount0,  
deadline);  
  
permit2.approve(Currency.unwrap(key.currency1), address(positionManager), amount1,  
deadline);
```

Impact

There are two possible scenarios;

- Some ERC-20s return `false` rather than reverting, ignoring the return value makes failures show up later and harder to diagnose.
- Some tokens like USDT require setting allowance to zero before changing it from one non-zero value to another, repeated launches can become permanently broken for those tokens.

Recommendation

Use a SafeERC20-style approach (check return values, handle non-standard tokens).

Status

Resolved

Medium

[M-01] GenesisTokenFactoryV1#__launchTokenEconomy - POD ownership not enforced on-chain

Description

The factory does not verify POD existence or ownership on-chain during economy creation; it assumes the trusted launcher only submits valid (podId, podOwner) pairs.

Vulnerability Details

__launchTokenEconomy only checks _podId range, and that an economy doesn't already exist, it does not call an ERC-721 ownerOf. eip712LaunchEconomy only checks podOwner == msg.sender. Making the correctness an operational invariant, if the trusted launcher ever supplies an incorrect mapping, the wrong mapping is written permanently because subsequent launches revert with EconomyExists().

Impact

It can cause a permanent denial of service depending on a failure or compromise of the trusted launch infrastructure.

Recommendation

Add an on-chain ownerOf(podId) check against the POD ERC-721 contract.

Status

Acknowledged

[M-02] GenesisTokenFactoryV1#__createDexLiquidityPool - USDC decimal normalization underflow can revert dual-pool launches

Description

Dual pool launch can revert when the economy token decimals are less than the USDC decimals.

Vulnerability Details

`decimalsAdjustment` is computed as a subtraction into `uint8`. If the economy token uses fewer decimals than USDC (e.g. a six-decimal economy token), this subtraction underflows, and the entire dual-pool launch reverts.

Impact

Operational DoS for dual liquidity pool creation under certain token decimal configurations.

Recommendation

Handle both directions explicitly, multiply or divide based on the sign of the decimal difference.

Status

Resolved

Low

[L-01] UniswapV4SwapFeeHookV1#_beforeSwap/_afterSwap - Fee rounding allows zero-fee micro-swaps

Description

For sufficiently small swaps, the computed fee rounds to zero. In some market conditions, a trader can reduce fees by splitting into many small swaps (bounded by gas, ME,V and worse execution).

Recommendation

Consider a minimum fee floor and/or dust accumulation if fee integrity is important.

Status

Acknowledged

[L-02] GenesisTokenFactoryV1#__deployToken - `genesisTokenImpl` is not validated as contract code

Description

The factory clones `genesisTokenImpl` without validating that it is actually a deployed contract (non-empty code) and compatible with expected initialization behavior.

Recommendation

Validate `genesisTokenImpl.code.length > 0` and optionally validate expected interface/version prior to cloning.

Status

Resolved

[L-03] GenesisTokenFactoryV1#__createDexLiquidityPool - Unverified
`_aliPriceInUSD` can misprice treasury-seeded liquidity

Description

The initial LP seeding ratio uses `_aliPriceInUSD` supplied by a privileged launcher/signed request. The contract does not sanity-check this value on-chain: the trusted process must provide an accurate price. If that process is wrong, the pool can be seeded at a materially wrong price.

Recommendation

Treat `_aliPriceInUSD` as a trusted input and document it as such, add guardrails (like bounded ranges vs a reference). If you want to enforce the invariant on-chain, validate against an oracle.

Status

Acknowledged

QA

[Q-01]

GenesisTokenFactoryV1#updateTokenShareConfig/updateV4HookFeeConfig - BPS documentation inconsistent with implementation

Description

Some comments reference `100_000 bps = 100%`, but the implementation consistently uses `10_000` as `100%`.

Recommendation

Fix comments to match implementation or update implementation if `100_000` is intended.

Status

Resolved

[Q-02] GenesisTokenFactoryV1#eip712LaunchEconomy - EIP-712 validity window semantics mismatch

Description

Implementation uses strict bounds (`block.timestamp > validAfter` and `< validBefore`), while the field comments can be read as inclusive.

Recommendation

Align comments and code semantics. If inclusive bounds are desired, adjust comparisons accordingly.

Status

Resolved

[Q-03] UniswapV4SwapFeeHookV1#getHookPermissions - Hook permission comments contradict behavior

Description

Comments in the hook permissions mention “exact-input only,” but the hook implements both exact-in and exact-out fee logic paths.

Recommendation

Update comments to reflect actual cases handled.

Status

Resolved

[Q-04] V4HookFeeDistributorV2#IGenesisTokenFactory.getPodId - Interface signature mismatch

Description

Interface in fee distributor defines `getPodId` as non-view and returns `uint256`, while the factory implementation is view and returns `uint32`.

Recommendation

Match interface signature to implementation: external view returns (`uint32`).

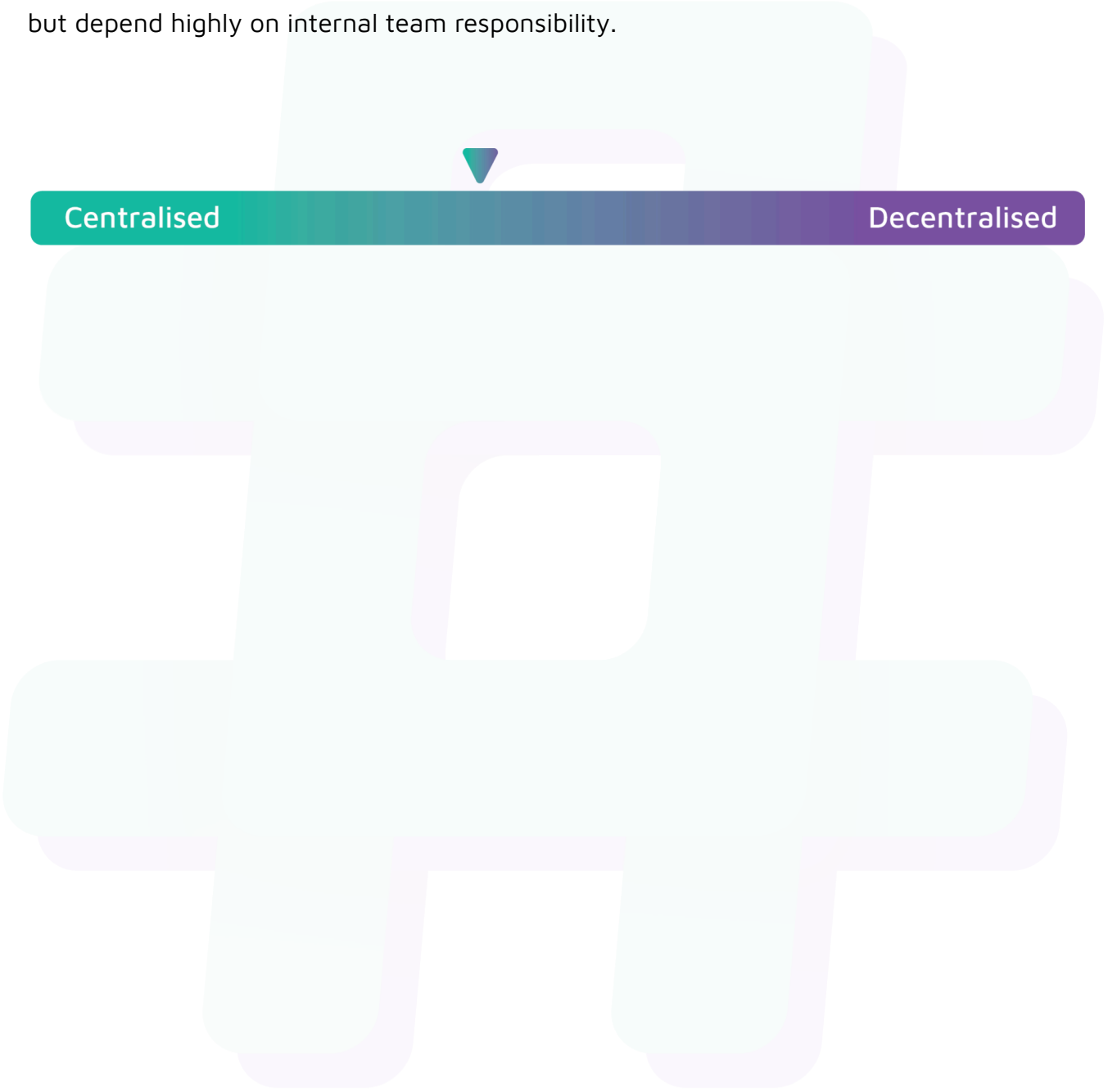
Status

Resolved

Centralisation

The AI Protocol project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the AI Protocol project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.